

09

Agent Memory, Tools & MCP

Building Production AI Agents

IN THIS WEEK · 3 TOPICS · 35 SECTIONS

01

Memory Systems

10 SECTIONS

- 01 Short-term Memory
- 02 Long-term Memory
- 03 Episodic Memory
- 04 Semantic Memory
- 05 Conversation Memory
- 06 Knowledge Memory
- 07 Vector Memory
- 08 Memory Compression
- 09 Memory Retrieval
- 10 Memory Persistence

02

Tool Calling

12 SECTIONS

- 01 Function Calling
- 02 Tool Selection
- 03 Structured Outputs
- 04 API Integration
- 05 REST Tools
- 06 Database Tools
- 07 Search Tools
- 08 Browser Tools
- 09 File System Tools
- 10 Code Execution
- 11 Error Recovery
- 12 Retry Strategies

03

Model Context Protocol (MCP)

13 SECTIONS

- 01 MCP Fundamentals
- 02 MCP Architecture
- 03 MCP Client
- 04 MCP Server
- 05 MCP Resources
- 06 MCP Prompts
- 07 MCP Tools
- 08 Transport Layer
- 09 stdio Transport
- 10 SSE Transport
- 11 Authentication
- 12 Security
- 13 Enterprise MCP Design

WEEK 09 OF 17

01

02

03

04

05

06

07

08

09

10

11

12

13

14

15

16

17

Memory Systems

01 Short-term Memory

Short-term memory refers to the information an agent needs to keep track of just within its current, ongoing task or conversation — everything that's happened so far in this particular session, held onto just long enough to be useful for the immediate work at hand, without necessarily needing to be remembered forever afterward.

Think about what this actually includes in practice: the original goal or question the user gave the agent, every action the agent has taken so far during this task, every result or observation that came back from those actions, and the general flow of back-and-forth that's happened up to this exact moment. Without this, an agent working through a multi-step task would be a bit like someone trying to solve a puzzle while suffering from amnesia every thirty seconds — it might repeat an action it already tried, contradict something it said just a moment earlier, or completely lose track of what it was originally even trying to accomplish partway through.

In practice, for most language-model-based agents, short-term memory is typically implemented in a fairly direct way — by keeping a running record of everything that's happened so far in the current task, and including that entire record as part of what gets sent to the language model with each new step. This connects directly back to something we discussed in the earlier LLM API explanation: since a language model has no memory of its own between separate calls, the "memory" has to be recreated each time by literally resending the relevant history along with the new request. The obvious limitation here is the context window we've discussed before — there's a maximum amount of text a model can process in one go, so short-term memory can't just grow forever without eventually running into that ceiling, which is exactly why techniques like memory compression (which we'll get to shortly) become important for longer-running tasks.

02 Long-term Memory

Long-term memory is the counterpart to short-term memory, and it refers to information an agent retains and can draw on well beyond just the current task — persisting across entirely separate conversations, sessions, or even much longer stretches of time, sometimes indefinitely.

The distinction matters a lot in practice. Imagine you're using a personal AI assistant regularly over many weeks. Short-term memory would let it keep track of what you're discussing right now, within today's conversation. Long-term memory is what would let it remember, days or weeks later, that you mentioned you're vegetarian, or that you prefer concise answers, or that you're currently working on a specific ongoing project — details that were shared at some earlier point, potentially in a completely separate conversation, but that remain genuinely useful and relevant going forward.

Unlike short-term memory, which can reasonably just be "everything that's happened so far in this session," long-term memory can't work the same way, because the total volume of everything an agent has ever encountered across all of its interactions would very quickly become far too large to include in full with every single new request — it would blow past any context window almost immediately, and most of it wouldn't even be relevant to what's currently being discussed anyway. Because of this, long-term memory systems generally need a more deliberate approach: information worth remembering gets stored somewhere

persistent (often in a vector database, which we'll connect to shortly), and then, when it's actually needed, only the specific relevant pieces get retrieved and pulled back in, rather than the agent trying to carry its entire accumulated history around with it at all times.

03 Episodic Memory

Episodic memory refers to an agent remembering specific past events or experiences — particular things that happened, at a particular point in time, often including details about the surrounding context of that specific occurrence. The term is actually borrowed directly from human psychology, where "episodic memory" describes your ability to recall a specific personal experience, like remembering the actual event of your first day at a new job, as opposed to just abstractly knowing a general fact.

For an AI agent, episodic memory might look like remembering that "on this particular date, the user asked for help planning a trip to Japan, and here's the specific sequence of things we discussed and decided," or "the last time I tried to use this particular tool with these particular inputs, it failed with this specific error." This is meaningfully different from just knowing a general fact — it's remembering an actual specific occurrence, complete with its own particular details and context, that happened at a specific point in the agent's history.

This kind of memory tends to be genuinely valuable for a few practical reasons. It lets an agent learn from its own specific past experiences — if a particular approach failed in a very similar past situation, recalling that specific episode can help the agent avoid repeating the same mistake. It also supports genuine continuity in an ongoing relationship with a user — being able to reference "last time we talked about this, you mentioned..." makes an agent feel meaningfully more attentive and personalized, rather than treating every single interaction as a completely isolated, disconnected event with no sense of an ongoing shared history.

04 Semantic Memory

Semantic memory, again borrowing its name directly from human psychology, refers to general knowledge and facts that an agent knows or has learned — not tied to any one specific past event or experience, but rather abstracted, general knowledge that holds true across many different situations.

To make the distinction with episodic memory really clear, consider this comparison: episodic memory would be "I remember the specific conversation last Tuesday where the user told me they're allergic to peanuts." Semantic memory would be the resulting general fact itself, now detached from that specific originating conversation: "this user is allergic to peanuts." The specific episode where that fact was learned might eventually fade in relevance or even get discarded entirely, but the extracted, general fact itself remains and continues to be useful going forward, independent of the specific original moment it came from.

In practice, well-designed agent memory systems often deliberately extract semantic memory out of episodic experiences over time — noticing that a particular fact or preference has come up, pulling it out as a standalone, general piece of knowledge, and storing it in a way that's easy to retrieve and apply later, separate from all the specific surrounding conversational detail of exactly when and how it was originally mentioned. This mirrors how human memory itself tends to work too — you probably don't remember the exact specific moment and conversation where you first learned that your friend doesn't like coffee, but you retain and use that general fact about them going forward regardless, long after the specific memory of that original conversation has faded.

Conversation Memory

Conversation memory refers specifically to an agent's ability to keep track of the back-and-forth history of an ongoing dialogue with a user — remembering what's been said so far in this particular exchange, so that later messages can build naturally and coherently on earlier ones, rather than each new message being treated as a completely fresh, disconnected interaction with no awareness of what came before it.

This is genuinely one of the most immediately noticeable and important forms of memory for anyone actually using an AI system in a conversational way. Without it, if you said "my name is Alex" in one message and then asked "what's my name?" in your very next message, the AI would have no way of answering correctly, because it would have no memory whatsoever of your first message by the time it received your second one. Conversation memory is what allows for genuinely natural, flowing dialogue, where you can reference something you said a few messages ago, ask a follow-up question without having to fully restate all the necessary context every single time, or have the AI gradually build up an increasingly complete and accurate understanding of what you're trying to accomplish as the conversation naturally continues and develops.

In its simplest and most common form, conversation memory is implemented by keeping the full transcript of the conversation so far, and including that entire transcript as part of what gets sent to the language model with each new message, exactly as we described in the short-term memory section above. For very long conversations, though, this simple approach can eventually bump into the practical limits of the context window we've discussed several times now, which is where techniques like summarizing or otherwise compressing older parts of the conversation become genuinely necessary to keep things manageable.

Knowledge Memory

Knowledge memory refers to a broader store of factual, reference-type information that an agent can draw on — not necessarily tied to any specific conversation or specific user at all, but rather general, reusable knowledge relevant to whatever domain the agent operates in.

This connects very directly and naturally back to everything we discussed in the RAG explanations earlier in this series. A customer support agent's knowledge memory might be an entire company's product documentation and policy manuals. A coding assistant's knowledge memory might be a codebase's documentation, or a library's technical reference material. A research assistant's knowledge memory might be a curated collection of relevant academic papers or trusted reference sources. This is genuinely the same underlying idea as the "knowledge base" we discussed extensively in the RAG explanations — it's knowledge memory specifically viewed and understood through the lens of an agent's overall memory system, rather than as a completely separate, unrelated concept.

The important distinction to hold onto here is that knowledge memory is generally more static and shared than something like conversation memory or episodic memory — while a specific user's conversation history or personal preferences are unique to that individual, an agent's knowledge memory (like a shared company manual, or reference documentation) is typically the same underlying resource that gets drawn upon and reused across many different users and many different conversations, rather than being personal or unique to any one specific individual's ongoing interaction history with the agent.

Vector memory refers to a specific, extremely common way of technically implementing long-term memory (and often episodic, semantic, and knowledge memory too) using exactly the vector embeddings and vector databases we covered in detail in the earlier explanations on RAG foundations and vector databases.

Here's how the pieces connect together. Instead of storing an agent's accumulated memories as a simple, growing list of plain text that just keeps getting longer and longer over time, each individual memory (a fact, a past conversation snippet, a learned preference) gets converted into a vector embedding — that numeric representation of its meaning, exactly as we discussed earlier — and stored in a vector database. Then, whenever the agent actually needs to recall something relevant to its current situation, it doesn't have to scan through absolutely everything it's ever learned; instead, it converts its current context or question into a vector too, and performs the same kind of similarity search we discussed at length in the RAG explanations, quickly retrieving just the specific memories that are actually most relevant to what's happening right now.

This is genuinely one of the most practical and widely used solutions to the fundamental challenge we identified earlier with long-term memory — the simple fact that an agent's total accumulated history over time can become far too large to ever include in full with every single request. Vector memory elegantly sidesteps this problem by allowing the agent to efficiently and quickly search through a potentially enormous amount of accumulated history, and pull back only the small handful of specific, genuinely relevant memories that actually matter for the current moment — essentially applying the exact same "search for what's relevant rather than including everything" philosophy that underpins RAG in general, just now applied specifically to an agent's own personal memory and accumulated history, rather than to a separate external document collection.

Memory compression refers to the set of techniques used to shrink down accumulated memory — whether that's a long ongoing conversation, or a large collection of past episodic memories — so that it takes up meaningfully less space, while still retaining the most genuinely important and useful information.

Why is this necessary at all? We've now run into the same underlying tension repeatedly throughout this whole explanation: context windows are limited in size, and including too much raw, uncompressed information isn't just expensive in terms of cost (remember, you're generally billed based on tokens, as we discussed in the earlier LLM APIs explanation), it can also genuinely make an agent perform worse, since important details risk getting buried and effectively lost among a large amount of less critical accumulated history.

A few common approaches are used to tackle this. **Summarization** is probably the most straightforward and widely used technique — periodically, instead of keeping the full, exact, word-for-word history of everything that's happened so far, the system generates a condensed summary that captures the key points and important details, and then continues going forward using that shorter summary in place of the full original detail, freeing up a meaningful amount of space while still preserving the genuinely essential information.

Selective retention is another approach, where instead of summarizing absolutely everything indiscriminately, the system makes more deliberate, active decisions about which specific pieces of information are actually worth keeping around at all (based on relevance, importance, or how recently they came up), and simply discards or lets go of the rest entirely, rather than trying to preserve some compressed version of everything regardless of its importance. **Hierarchical compression** takes this idea a step further, applying increasingly aggressive compression the further back in time something goes — recent events

might be kept in full, complete detail, while older events get progressively more heavily summarized and condensed the further back into the past they recede, mirroring, in a rough way, how human memory itself also tends to naturally retain vivid, detailed memories of very recent events while gradually blurring and generalizing much older ones over time.

The core underlying goal across all of these different compression techniques is the same: preserving as much of the genuinely useful "signal" as possible, while shedding as much of the less important, space-consuming "noise" as you reasonably can, so an agent's memory stays both manageable in size and still genuinely useful for informing its ongoing decisions and behavior.

09 Memory Retrieval

Memory retrieval refers to the actual process of pulling the right, relevant memories back out at exactly the moment they're genuinely needed — which is really the natural and necessary complement to everything we've discussed so far about storing and compressing memory in the first place. Having a large, well-organized, well-compressed store of memories is only genuinely useful if the agent can reliably find and surface the specific right pieces at the right moment when they're actually needed.

This connects very directly back to the vector memory and similarity search concepts we covered above and in the earlier RAG explanations — in practice, memory retrieval very often works by converting the agent's current situation or question into a vector, and then searching the stored memory for the pieces whose embeddings are most similar in meaning, exactly like we described with semantic search generally. But retrieval can also draw on other useful signals beyond pure semantic similarity alone — for example, weighting more **recent** memories somewhat more heavily than very old ones (since more recent context is often, though certainly not always, more relevant to the current moment), or weighting memories that have proven to be particularly **important** or frequently useful in the past more heavily than ones that have rarely, if ever, actually turned out to matter.

Getting memory retrieval genuinely right is a real and non-trivial design challenge, and it's worth appreciating why. Retrieve too little, or retrieve the wrong specific memories, and the agent effectively behaves as if it never actually had that stored information in the first place, even though it was technically sitting there in storage the whole time, simply because it never actually got surfaced and used at the moment it was actually needed. Retrieve too much, or retrieve a lot of irrelevant, low-value memories alongside the genuinely useful ones, and you run straight back into the same context-window bloat and "signal buried in noise" problems that compression was specifically trying to solve in the first place. Well-designed memory retrieval systems are constantly trying to strike a careful, deliberate balance between these two competing failure modes.

10 Memory Persistence

Memory persistence refers to actually ensuring that an agent's memory genuinely survives and remains available over time — rather than being lost the moment a particular session ends, a program restarts, or the underlying computer running the agent gets shut down and later started back up again.

This might sound like an obvious, almost trivial requirement, but it's actually a real and important practical engineering consideration, and it's worth understanding why it doesn't just automatically happen on its own. If an agent's memory is only ever held in temporary, working computer memory (often referred to as RAM) during a session, and never gets specifically and deliberately saved somewhere more durable, then all of that accumulated memory simply vanishes completely the moment that particular session or process ends —

meaning the agent would effectively wake up with total, complete amnesia every single time it's started back up again, no matter how much useful, valuable context and information it had previously accumulated during earlier sessions.

Achieving genuine memory persistence means deliberately saving an agent's memory to some form of durable, longer-term storage — this is very often the vector database we discussed at length earlier (for long-term, semantic, and episodic memory), but it can also include more traditional databases for storing more clearly structured facts, or simply saving data to files on disk for smaller and simpler systems. This durable storage needs to reliably survive restarts, crashes, and any updates made to the underlying system running the agent, so that when the agent starts back up again later, it can reliably reload and reconnect with everything it had previously learned and stored, rather than starting over again completely from scratch. For any genuinely production-grade agent system — one that's actually meant to provide real, ongoing value to real users over an extended period of time, rather than existing purely as a short-lived, one-off demonstration or experiment — reliable memory persistence is a genuinely essential, foundational requirement, not an optional nice-to-have feature to consider adding on only later, as an afterthought, once everything else about the system is already working well.

Tool Calling

01 Function Calling

Function calling is the foundational mechanism that makes tool use possible in the first place, so it's worth understanding clearly before anything else in this section. At its heart, the idea is this: instead of a language model only being able to respond with plain, free-form text, it can also be given the ability to respond by saying, in a very specific, structured way, "I want to call this particular function, with these particular input values."

Here's how this actually works in practice. When you set up an agent, you describe to the language model, in advance, exactly what functions (also often called "tools") it's allowed to use — giving each one a clear name, a description of what it does, and a precise specification of what inputs it needs (for example, a "get_weather" function might need a "city name" as its input). Then, during the conversation, if the model determines that using one of these functions would actually help answer the current question, instead of just generating a plain text reply, it generates a specially structured response indicating exactly which function it wants to call and with what specific input values. The surrounding system (not the model itself) then actually executes that function in the real world, takes whatever result comes back, and feeds that result back to the model, which can then use it to continue forming its final answer.

It's worth being really clear about an important detail here, because it trips up a lot of beginners: the language model itself never directly runs any code or reaches out onto the internet on its own. It only ever generates text — but a very specific, structured kind of text that clearly says "please call this function with these inputs." It's entirely up to the surrounding application code to actually notice that structured request, genuinely go carry out the real action, and then report the result back. This separation matters a lot for safety and control — it means a developer always has a deliberate checkpoint where they can inspect, validate, or even block a requested function call before anything real actually happens, rather than the model having some kind of direct, unsupervised, unchecked ability to act in the world entirely on its own.

02 Tool Selection

Tool selection refers to the process — and the real underlying challenge — of an agent correctly figuring out which specific tool, out of possibly many available ones, is actually the right one to use for a given situation, and this becomes a genuinely harder problem than it might sound like once an agent has more than just a couple of tools available to it.

Think about it from the model's perspective: if an agent has been given access to twenty different tools (a web search tool, a calculator, a database lookup tool, a weather checker, a calendar tool, and so on), and a user asks a somewhat ambiguous question, the model needs to correctly reason about which of those twenty tools, if any, is actually the right fit for addressing that particular request — and it needs to get this right fairly reliably, because picking the wrong tool, or failing to recognize that a tool is needed at all, directly leads to a poor or completely wrong final answer.

A few practical things genuinely help make tool selection work more reliably in real systems. **Clear, well-written tool descriptions** matter enormously — a tool that's vaguely or ambiguously described (like a function simply named "lookup" with a one-word description) gives the model far less to work with when

deciding whether it's actually the right fit, compared to a tool with a clear, specific name and a precise, well-written explanation of exactly what it does and doesn't do. **Not overwhelming the model with too many tools at once** also matters — much like a human faced with an overwhelming wall of options, giving a model access to an excessive number of tools simultaneously, especially ones with overlapping or unclear purposes, tends to genuinely hurt its ability to reliably pick the correct one, which connects directly back to the Router pattern we discussed in the earlier Agent Patterns explanation, where narrowing down the relevant set of available tools or paths ahead of time can meaningfully improve overall reliability. And **good examples and clear instructions** in the surrounding prompt can further help guide a model toward correctly recognizing which situations genuinely call for which specific tool.

03 Structured Outputs

Structured outputs refers to getting a language model to produce its response in a precise, predictable, well-defined format — like a specific structure with clearly named fields and specific data types — rather than in loose, free-flowing natural language text that a program would then have to try to interpret or parse afterward.

Why does this matter so much for tool calling specifically? Because for a tool call to actually work correctly, the surrounding application code needs to reliably know exactly what input values the model is providing, in a format the code can directly and confidently use — it can't be left guessing or trying to loosely interpret something like "the model seems to want to search for something related to Paris weather, I think."

Structured outputs solve this by having the model's response conform to a specific, predefined structure (very often something like a JSON format, which is a very common and simple way of representing structured data), so the exact input the model wants to send to a function is clearly, precisely, and unambiguously specified — for example, unambiguously specifying `{"city": "Paris", "units": "celsius"}` rather than a vague, informally-worded sentence that a program would then have to try to interpret.

This capability isn't just useful for tool calling — it's also broadly valuable any time you want a model's output to reliably plug directly into some other piece of software without a human having to manually read and interpret free-form text first. Many modern language model APIs offer specific, dedicated features to help enforce this reliably, letting a developer specify the exact expected structure in advance and getting a strong, often technically enforced guarantee that the model's response will actually conform to that structure, rather than just hoping the model happens to format its text response correctly through good instructions alone.

04 API Integration

API integration refers to the general practice of connecting an agent to external APIs — the standard interfaces that different software services and companies expose, allowing separate pieces of software to communicate with each other and exchange data or trigger actions.

You've actually already learned a lot of the underlying concept here from the very first explanation in this whole series, on LLM APIs — an API, in general, is simply a defined, standardized way for one piece of software to request something from another piece of software over the internet, and get a structured response back. Tool calling, in a very real sense, is largely about giving an agent the ability to make exactly

these kinds of API calls itself (through the function-calling mechanism we described above), rather than only a human developer manually writing code that calls a fixed, predetermined API at a fixed, predetermined moment.

In practice, integrating an agent with external APIs generally involves a few concrete pieces of work: defining a clear tool description that accurately explains what a particular API actually does and what inputs it specifically needs, handling authentication properly (many APIs require some kind of secret key or token to prove you're an authorized, legitimate user, and this needs to be securely managed and never exposed improperly), and handling the response that comes back in a way the agent can actually make good use of (which sometimes means the raw response needs some reformatting or simplification before being handed back to the model, since raw API responses can occasionally be quite verbose, complex, or contain a lot of extra technical detail the model doesn't actually need). Good API integration is really about creating a clean, reliable, well-documented bridge between the agent's more abstract, flexible reasoning capability and the very concrete, specific, structured way that real external services and systems actually expect to be interacted with.

05 REST Tools

REST is one of the most common and widely used standard styles for designing APIs across the software industry, and understanding it a little will help make sense of a huge number of the specific tools an agent might actually be given access to in practice.

Without going too deep into the technical weeds, REST is built around a few simple, consistent underlying ideas. It organizes everything around "resources" — specific pieces of data or things you can act upon, like "a customer," "an order," or "a product," each typically identified by its own specific web address. And it uses a small, standard, consistent set of actions to interact with those resources — most commonly: fetching or reading a resource (called "GET"), creating a brand new resource (called "POST"), updating an existing resource (called "PUT" or "PATCH"), and removing a resource entirely (called "DELETE"). Because so many different companies and services have all generally settled on and adopted this same consistent underlying style, once you understand this basic pattern, a huge number of different real-world APIs across a huge range of different services all start to feel genuinely familiar and follow broadly the same general shape and logic.

When we talk about "REST tools" specifically in an agent context, we simply mean tools that are built to call these kinds of standard REST-style APIs on the agent's behalf — for example, a tool that lets an agent fetch a customer's order history by calling a company's REST API, or a tool that lets an agent create a new support ticket by sending a properly formatted request to a REST endpoint designed specifically for that purpose. Because REST is such a widely adopted, well-understood, and consistent standard across the software industry, building tools that wrap and expose REST APIs to an agent is one of the single most common and foundational ways of actually connecting an agent to real, useful, external capabilities in the world.

06 Database Tools

Database tools give an agent the ability to directly read from, and sometimes write to, a structured database — letting it look up specific stored records, run queries to find and retrieve particular pieces of information, or, in some cases, actually create, update, or delete data.

This is an especially powerful and valuable category of tool, because a huge amount of the world's genuinely useful, day-to-day business information lives in structured databases rather than in loose documents or general web pages — think of a company's customer records, their product inventory, their order history, or their internal analytics data. Giving an agent the ability to directly query this kind of information means it can answer very specific, precise, factual questions ("how many units of this specific product do we currently have in stock?") that a general knowledge model, or even a document-based RAG system, would have absolutely no reliable way of answering on its own.

Database tools do come with some genuinely important design considerations worth knowing about, precisely because they can be significantly more powerful, and correspondingly significantly more risky, than a simple, harmless read-only search tool. **Read versus write access matters enormously** — a tool that only lets an agent look up and read existing information is generally much lower-risk than one that lets it actually modify or delete real, live data, and many well-designed production systems deliberately restrict agents to read-only database access specifically to avoid the very real risk of an agent, whether through a genuine reasoning mistake or an unexpected edge case, accidentally corrupting or deleting important, valuable data. **Query safety also matters a lot** — since agents ultimately construct their own database queries based on their own reasoning, well-designed systems need real, deliberate safeguards to prevent an agent (or a malicious actor deliberately trying to manipulate it through crafted, adversarial user input) from being able to construct a harmful, overly broad, or dangerously unrestricted query that exposes or damages far more data than was ever actually intended.

07 Search Tools

Search tools give an agent the ability to look up current, external information — most commonly by actually searching the web, though this general category can also include searching within a specific, more narrowly defined document collection or knowledge base, connecting directly back to the semantic search and RAG concepts we covered in detail earlier in this whole series.

This is genuinely one of the single most common and valuable tools given to agents, and it directly addresses one of the core motivating problems behind RAG that we discussed way back at the start of this whole series: a language model's own built-in training data has a fixed cutoff date, and it simply has no way of knowing about anything that's happened more recently, or about anything genuinely current, like today's news, current prices, or the current status of something happening right now. A search tool lets the agent reach outside its own frozen training knowledge and pull in fresh, current, real information at the actual moment it's needed, rather than being limited entirely to whatever it happened to learn during its original training process.

In practice, a well-designed search tool typically takes a text query as its input (which the agent itself formulates, based on what it's actually trying to find out), sends that query off to an actual underlying search engine or a specific, defined knowledge source, and returns a set of relevant results back to the agent — which the agent then needs to actually read through, interpret, and meaningfully incorporate into its final response. This connects very directly back to a number of the advanced RAG techniques we covered earlier, like query transformation (rewriting a vague or poorly-worded question into a more effective search query before actually running the search) and re-ranking (further refining and improving a batch of initial search results before actually using them) — these same underlying techniques apply just as well and just as usefully here, within the specific context of an agent actively deciding, on its own, exactly when and how to search for information as part of accomplishing some larger, broader task.

Browser tools give an agent the ability to actually control and interact with a real web browser — not just fetching and passively reading the raw content of a single webpage (which a simpler search or page-fetching tool can already do), but genuinely being able to click on links and buttons, fill out and submit forms, scroll down a page to reveal more content, and navigate from page to page across a website, much like a human actually would while manually browsing the web themselves.

This meaningfully expands what an agent can actually accomplish out in the real world, well beyond what a simple search tool alone would allow. A search tool can help an agent find and read relevant information — but a browser tool lets an agent actually take real action on a live, interactive website: filling in and submitting a genuine contact form, actually clicking "add to cart" on a specific product, logging into an actual account, or working through a live, multi-step process on a website that genuinely requires several sequential interactions to complete, rather than the agent being limited only to passively reading whatever static content happens to already be sitting there on a page.

Browser tools tend to come with some genuinely real added complexity, though, that's worth understanding clearly. Modern websites are often quite complex and highly dynamic, meaning an agent controlling a browser needs a genuinely reliable way to correctly identify and interact with the right specific elements on a given page (the right specific button, the right specific input field), even as a page's underlying layout and structure might change or update over time. There's also a real, meaningfully elevated safety consideration here compared to a simple, purely read-only search tool — since a browser tool can potentially take genuinely consequential, real-world actions (submitting a real, live purchase, changing real account settings, and so on), the kind of human-in-the-loop safeguards we discussed earlier in the Agent Patterns explanation become especially important and especially relevant specifically when an agent has this level of real, powerful, interactive browser capability available to it.

File system tools give an agent the ability to interact directly with files — reading the actual contents of an existing file, creating a brand new file, modifying or editing an existing file's contents, or organizing and managing files and folders more generally.

This category of tool becomes especially important and especially relevant for agents that work with actual documents or code as a core, central part of their job — for example, a coding agent that genuinely needs to read through existing source code files, actually make specific edits directly to them, and then create new files as part of implementing some requested feature or fix, or a document-processing agent that needs to actually read through, meaningfully summarize, and organize a large batch of files into some more useful and manageable structure.

As with database tools, the specific scope of exactly what file system access an agent is actually granted matters a great deal in practice. A well-designed production system generally deliberately limits and clearly scopes exactly which specific files and folders an agent can actually access and modify (for example, restricting it to only a specific, designated project folder, rather than granting it broad, unrestricted access to an entire computer's complete file system), specifically to prevent an agent from accidentally reading or modifying something sensitive, important, or entirely unrelated to its actual, intended task, whether that mistake arises from a genuine reasoning error on the agent's part, or from a deliberately crafted, adversarial attempt by a malicious user to manipulate the agent into taking an unintended, harmful action outside its properly intended scope.

Code execution tools give an agent the ability to actually write and run real, executable code, and then directly see the genuine results of running that code — rather than only being able to reason abstractly about what a piece of code would probably do, purely through its own internal language-based reasoning, without ever actually being able to verify that reasoning against real, concrete execution.

This is a genuinely powerful capability, and it's worth understanding clearly why it matters so much. Language models, on their own, aren't fundamentally calculators or code interpreters — they're generating their text output based on learned patterns from their training data, which means they can sometimes make small logical or arithmetic mistakes, especially on more complex, multi-step calculations, purely because they're reasoning about the answer in the abstract rather than actually, concretely computing it. Giving an agent the ability to actually write and run real code changes this dynamic substantially — instead of trying to solve a complex math problem, or process a large, complicated dataset, entirely "in its head" through pure language-based reasoning alone, it can write an actual, real script to genuinely perform that calculation with full precision, and directly, empirically observe the real, verified result, rather than just confidently generating what merely sounds like a plausible answer without any way to actually verify it's correct.

Code execution tools are typically run inside a deliberately sandboxed, carefully isolated environment — a genuinely restricted, contained space specifically designed to prevent code the agent writes and runs from being able to cause any real, unintended damage to the broader, actual underlying system, or from being able to access things outside its own narrow, specifically intended scope. This sandboxing matters enormously for safety, precisely because code execution is inherently one of the most powerful and potentially high-risk categories of tool an agent can be given — genuinely arbitrary code, if left completely unrestricted, could in principle do almost anything at all on the underlying computer running it, so carefully containing and limiting exactly what that code is actually capable of doing and reaching is a genuinely critical design consideration.

Error recovery refers to how well an agent handles things going wrong during a tool call — because in any genuinely real-world system, things absolutely do go wrong with real regularity: a website might be temporarily unreachable, an API might return an unexpected error, a search might turn up no genuinely useful results at all, or a piece of code the agent wrote might have a bug and fail to actually run correctly.

A poorly designed agent simply breaks down completely the moment something unexpected happens — it either crashes outright, or it gets confused and effectively stuck, having no sensible, well-defined way of actually handling or working around the specific problem it just ran into. A genuinely well-designed agent, by contrast, is specifically built to anticipate that these kinds of failures will inevitably happen sometimes, and to actually handle them gracefully when they do, rather than treating every single failure as an unrecoverable, catastrophic dead end.

Good error recovery generally involves a few specific, practical things working together. The agent needs to be able to **correctly recognize when an error has actually occurred in the first place** — clearly distinguishing a genuine, real failure from a normal, valid, successful result (for example, correctly recognizing that a tool call returning "no results found" is fundamentally different from that same tool call genuinely crashing with a technical error). It then needs to **reason sensibly about what to actually do next** in response to that specific error — perhaps trying a meaningfully different approach altogether, perhaps trying the exact same action again in case the failure was only a temporary, transient issue, or perhaps concluding

it genuinely can't currently complete this specific part of the task and honestly, clearly communicating that limitation back to the user, rather than either quietly giving up without any real explanation, or worse, confidently making something up to paper over the actual underlying failure. This connects very directly and closely back to the Reflection pattern we covered earlier in the Agent Patterns explanation — a well-designed agent genuinely reflects on whether its last action actually succeeded as intended, and specifically uses that honest reflection to meaningfully inform what it sensibly does next, rather than simply barreling straight ahead as though every single action along the way had necessarily gone perfectly and exactly as originally planned.

12 Retry Strategies

Retry strategies are a specific, more concrete piece of the broader error recovery puzzle we just discussed — specifically focused on the fairly common, practical question of exactly when and how an agent (or the surrounding system running it) should actually try a failed action again, rather than simply giving up entirely the very first time something doesn't work.

This matters because not every single failure is genuinely the same kind of failure, and treating every one of them completely identically often isn't actually the smartest, most effective way to handle things. Some failures are **transient** — meaning genuinely temporary, and quite likely to succeed just fine if you simply try again a short moment later (for example, a web service that's very briefly, temporarily overloaded with a burst of traffic, or a rare, one-off momentary network hiccup). Other failures are **persistent** — meaning trying that exact same failed action again, completely unchanged, in exactly the same identical way, is very unlikely to actually produce any genuinely different or better result the next time (for example, if a tool call has fundamentally, structurally invalid or malformed input to begin with, or if a user is straightforwardly asking for something that a particular available tool is simply, fundamentally not capable of doing at all, no matter how many times you retry it).

A well-designed retry strategy takes this important distinction into careful account, rather than treating every possible failure completely identically. Common, well-established practices include **only retrying failures that genuinely look transient** in nature, based on the specific type or nature of the error encountered, rather than blindly retrying absolutely everything without any real discrimination. Using something called "**backoff**," meaning waiting progressively longer between each successive retry attempt (rather than immediately and aggressively hammering away at the exact same failing action over and over again in rapid succession, which risks actually making an already-struggling underlying system's problems meaningfully worse) is another very common, well-established practice. And critically, **setting a firm, sensible limit on the maximum number of retry attempts** matters a great deal too — without some kind of sensible cap in place, a genuinely persistently and permanently broken action could otherwise cause an agent to get stuck endlessly and pointlessly retrying forever, burning through real time and real cost without ever actually making any genuine progress, rather than the agent eventually, sensibly recognizing that this particular specific approach clearly isn't working, and meaningfully moving on to try a genuinely different approach altogether, or honestly reporting the specific, persistent failure back to the user instead of silently looping forever.

Model Context Protocol (MCP)

01 MCP Fundamentals

At its core, MCP is an open protocol — meaning a publicly available, standardized set of rules — that defines a consistent way for AI applications to receive context and capabilities from external systems. The word "protocol" is doing important work here: much like how HTTP is the standardized protocol that lets any web browser talk to any website regardless of who built either one, MCP aims to be a standardized way for any AI application to talk to any external tool or data source, regardless of who built either side.

The problem MCP is specifically solving is often described using a simple, memorable analogy: before a common standard existed, connecting an AI application to N different external tools, while also wanting to support M different AI applications, meant potentially needing to build N times M different custom integrations — every tool needing its own special, one-off connection code for every different AI application that wanted to use it. This is sometimes compared to the " $M \times N$ problem." MCP solves this the same way a universal connector or a standard electrical outlet solves a similar problem in the physical world — once both sides agree to speak the same standard protocol, any MCP-compatible tool can be plugged into any MCP-compatible AI application without needing custom-built, one-off integration work for every single new pairing.

It's worth being clear about what MCP actually governs and what it doesn't. MCP is specifically about the protocol for exchanging context and capabilities between an AI application and external systems — it does not dictate how the AI application internally uses a language model or manages that context once it has it. It's a standard for the connection itself, not a standard for how the AI on the other end of that connection thinks or reasons.

02 MCP Architecture

MCP is built around a client-server architecture, which is a very common and well-established pattern throughout the software industry, so it's worth understanding the general shape of it before diving into MCP's specific pieces.

At a high level, there are three main roles involved. The **host** is the actual AI application the person is using — something like Claude Desktop, Claude Code, or another AI-powered tool. The host is the outer application that a real person actually interacts with. The **MCP client** lives inside that host application, and its job is to establish and manage a dedicated connection to one specific external system. Importantly, a host application creates a separate, individual MCP client for each different external server it wants to connect to — so if your AI application is connected to a file system tool, a database tool, and a search tool, it's actually running three separate MCP clients under the hood, each maintaining its own dedicated connection to its own corresponding server. The **MCP server** is the external piece — an independent, often fairly lightweight program that exposes a specific set of capabilities (specific tools, specific pieces of data, specific reusable prompt templates) through the standardized MCP interface.

A helpful way to picture the whole arrangement: the host application is like a person's phone, each MCP client is like one specific app installed on that phone (a banking app, a messaging app, a maps app), and each corresponding MCP server is like the actual remote service that app connects to and talks with behind

the scenes. The phone (host) can run several apps (clients) at once, each one maintaining its own separate, dedicated connection to its own separate remote service (server), without those different connections needing to know or care about each other at all.

03 MCP Client

The MCP client is the component that lives inside the host application and handles the actual mechanics of talking to one specific MCP server. Its job is fairly focused and specific: it establishes the connection to its corresponding server, handles the technical back-and-forth of the protocol itself (like properly formatted requests and responses), and essentially acts as the dedicated translator and messenger between the host application's needs and that one specific external server.

Each MCP client maintains what's called a one-to-one relationship with its server — meaning a single client is dedicated to talking with a single specific server, rather than one client trying to juggle connections to several different servers simultaneously. This clean separation is part of what makes the whole architecture modular and easy to reason about: if you want to add a brand new external capability to your AI application, you simply add a new MCP client-server pair, without needing to touch or modify any of the other existing connections your application already has set up.

From a beginner's perspective, it helps to think of the MCP client as something you generally don't interact with directly yourself — it's an internal, behind-the-scenes piece of plumbing inside whatever AI application you're actually using. As a user, you might see the effects of it (like an AI application successfully pulling information from your Google Drive or your GitHub account), but the actual client doing that connecting work is operating quietly under the hood.

04 MCP Server

An MCP server is the other end of the connection — an independent program that exposes a specific, focused set of capabilities to any MCP client that connects to it, following the standardized MCP protocol. Each server typically encapsulates one particular domain of responsibility — for example, a server specifically dedicated to interacting with a particular database, or a server specifically dedicated to providing access to a company's internal ticketing system, or a server specifically dedicated to GitHub repository access.

A really important design principle behind MCP servers is that they're meant to be modular and independently maintainable — the people who understand a particular external system best (say, the team who runs a particular company's internal database) can build and maintain the MCP server for that specific system, without needing to know or care about how any particular AI application chooses to actually make use of it. On the flip side, developers building an AI application don't need to understand the internal details of every different external system they want to connect to — they simply need their application to speak the standard MCP protocol, and it can then connect to any properly built MCP server, regardless of what's actually happening behind the scenes on that server's side.

MCP servers can run in a couple of different practical setups, which becomes important later when we talk about the transport layer — some run as local processes directly on the same computer as the host application (useful for things like accessing local files), while others run as remote services accessed over the internet (useful for things like connecting to a cloud-based company service).

05 MCP Resources

Resources are one of the core, standardized types of things an MCP server can expose to a connected client, and they specifically represent pieces of contextual data — things like documents, individual database records, file contents, or other pieces of information that the AI model might genuinely benefit from having access to as background context for the conversation.

Think of resources as answering the question "what information can this server make available for the AI to read and consider?" For example, a file-system-oriented MCP server might expose the contents of specific files as resources; a database-oriented MCP server might expose specific database records or query results as resources. This connects directly back to the RAG and knowledge-memory concepts we covered earlier in this whole series — resources are essentially a standardized way for an MCP server to say "here's some relevant data you might want to pull into your context," playing a very similar underlying role to the documents and chunks we discussed extensively in the RAG explanations, just now exposed through this consistent, standardized MCP interface rather than through a custom, one-off integration.

It's worth distinguishing resources conceptually from tools, which we'll cover next — resources are fundamentally about providing data and information for the AI to read and consider, whereas tools (discussed below) are about providing actions the AI can actually take and execute. Roughly speaking: resources are things to know, tools are things to do.

06 MCP Prompts

Prompts, as a specific MCP concept, refer to reusable, pre-defined prompt templates that an MCP server can expose to a connected client. Rather than a developer or user having to write out a well-crafted prompt from scratch every single time they want an AI to perform some particular common task well, a server can provide a ready-made, well-designed template for that specific task, which the host application can then surface and offer to the user.

Think of this as a kind of shared library of good, proven ways to ask an AI to do something specific and well-understood within a particular server's domain. For example, a server built around code review might expose a specific, carefully-crafted prompt template designed to guide an AI through performing a genuinely thorough, well-structured code review, rather than leaving it up to each individual user to figure out and write a good code-review prompt entirely on their own, from scratch, every single time they want one.

This particular MCP primitive is somewhat less central to everyday usage compared to tools and resources, but it serves a genuinely valuable purpose: it lets the people who deeply understand a particular domain (like the maintainers of a specific service or system) also share their accumulated expertise about how to actually prompt an AI effectively within that domain, packaged up and made conveniently available and reusable through the same standardized protocol, rather than that expertise living only informally in someone's head or in a scattered set of tips and tricks.

07 MCP Tools

Tools are very likely the most commonly used and most immediately impactful of MCP's core primitives, and they connect directly back to everything we discussed in the earlier Tool Calling explanation. An MCP server can expose specific, executable tools — like specific commands or specific actions — that the connected AI

application can actually invoke, following exactly the same fundamental function-calling mechanism we described earlier, just now delivered through this standardized MCP interface rather than through a custom, one-off integration.

This is genuinely the piece that turns MCP from being purely about providing background information into something that lets an AI application actually take real, concrete actions in the world. A GitHub-oriented MCP server, for example, might expose tools like "create a new issue," "search for a specific pull request," or "add a comment to an existing issue" — real, executable actions with clearly defined inputs, exposed in a standardized way that any properly connected MCP client can discover and correctly make use of.

A genuinely powerful aspect of this design is dynamic discovery — an MCP client doesn't need to have hardcoded, advance knowledge of exactly which specific tools a particular server offers. Instead, it can ask the server, at connection time, "what tools do you actually offer?" and the server responds with a clear, structured description of each available tool. This means a server's available tools can evolve and expand over time, and any properly connected client automatically becomes aware of new capabilities without needing to be specifically reprogrammed or updated to know about them in advance.

08 Transport Layer

The transport layer is the part of MCP's overall design responsible for actually physically moving messages back and forth between a client and a server — handling the real, practical mechanics of "how do these bytes of data actually get from one place to the other," separate and distinct from the "data layer," which defines what those messages actually mean and how they're structured.

This separation is a genuinely useful design choice, and it mirrors a similar separation you find in how the regular web works too — the meaning and structure of a web request is defined somewhat independently from the specific underlying network mechanism that actually carries it. By keeping these two concerns cleanly separated in MCP's design, the same core protocol (the same tools, resources, and prompts, structured in the same standardized way) can be delivered over a couple of genuinely different underlying transport mechanisms, depending on what actually makes the most practical sense for a given specific situation — whether the client and server are running on the very same computer, or are talking to each other across the internet.

Underneath both of the transport options we'll discuss next, MCP messages are structured using a well-established, standard messaging format called JSON-RPC 2.0 — a simple, widely used, easy-to-inspect and easy-to-debug way of formatting requests and responses as structured data. Understanding that this consistent, shared message format sits underneath both transport options helps make clear that switching between transports is really more about "how the message physically travels" rather than "what the message itself actually contains or means."

09 stdio Transport

stdio transport (short for "standard input/output") is the transport option specifically designed for situations where the MCP client and the MCP server are both running locally, directly on the very same computer. Rather than communicating over an actual network connection, they communicate directly through a computer's most basic input and output data streams — essentially, the client and server processes talk to each other extremely directly and locally, without any network involved at all.

This matters practically for a couple of genuine reasons. It's fast and simple, precisely because there's no actual network communication involved at all — everything is happening directly between two programs running on the same machine. And it's a particularly natural, well-suited fit for MCP servers that specifically need access to genuinely local resources — like a server that reads and writes to your local file system, or one that needs to interact with other software already installed directly on your own computer — since the server, in this setup, is quite literally running right there on your machine alongside the host application itself.

The tradeoff, of course, is that stdio transport is inherently a local-only mechanism — it fundamentally isn't designed for situations where the client and server need to communicate across the wider internet, connecting to some genuinely separate, remote computer or service running somewhere else entirely. For that broader kind of situation, a different transport option is needed, which brings us to the next topic.

10 SSE Transport

SSE stands for Server-Sent Events, and it refers to a transport option historically used by MCP specifically for situations where the client and server need to communicate over an actual network — meaning the server might be running remotely, on a completely different, separate machine somewhere else, rather than directly alongside the client on the very same computer.

The core idea behind SSE is that it allows a server to continuously push a stream of individual messages to a connected client over a standard, ordinary web connection, which is particularly well suited to the kind of ongoing, streaming communication that MCP interactions can often involve — for example, streaming back the individual, incremental pieces of a longer-running tool call's progress or output, rather than the client having to wait in total silence for one single, final, complete response to eventually arrive all at once.

It's genuinely worth understanding, as a beginner learning this space today, that MCP's transport story has continued to evolve since SSE was first introduced — a newer, generally more capable and more flexible transport approach called "Streamable HTTP" has increasingly become the standard, recommended way of doing remote, network-based MCP communication, effectively taking over much of the role that the original SSE-based approach used to serve. Since standards and specifications like this genuinely do keep evolving over time, if you're actually building something involving MCP transports today, it's worth double-checking the current official MCP documentation directly, rather than relying purely on what may become an increasingly outdated general understanding of exactly which specific transport mechanism is currently considered the standard, recommended approach for remote connections.

11 Authentication

Authentication refers to the process of an MCP client and server reliably verifying each other's identity, and specifically verifying that a particular client is actually genuinely authorized to access a particular server's capabilities and data, before any real communication or data exchange between them is actually allowed to proceed.

This matters enormously in practice, because MCP servers are very often gateways to genuinely sensitive systems and data — a company's private internal database, someone's personal email account, a company's confidential internal file storage. Without a genuine, properly implemented authentication mechanism in place, absolutely anyone who managed to connect to a server could potentially gain access to information or capabilities they were never supposed to have, which would obviously be a serious, genuinely unacceptable security problem.

For local, stdio-based connections, authentication is often less of a central practical concern, since the whole communication is already inherently confined and contained to a single local computer, and simply being able to run a program on your own machine in the first place already implies a meaningful degree of trust. For remote, network-based connections, though, proper authentication becomes considerably more critical and important, and MCP's design in this area draws on well-established, existing web authentication standards rather than inventing something completely new from scratch — a genuinely sensible design choice, since these existing standards have already been carefully tested, refined, and validated over many years of real-world, widespread use across the broader software industry.

12 Security

Security, considered more broadly than authentication alone, covers the full range of considerations involved in making sure that MCP connections and interactions can't be exploited or misused in genuinely harmful ways, extending well beyond the specific narrower question of simply confirming who's on each end of a given connection.

A few important, genuinely real security considerations are especially worth understanding in the MCP context specifically. **Scope and permissions** matter a great deal — a well-designed MCP server should generally only expose the genuinely specific, narrow set of tools and pieces of data that a given client actually, legitimately needs, rather than exposing overly broad, unnecessarily powerful access purely as a matter of convenience. **Validating inputs carefully** matters too — since a connected AI model is ultimately the one deciding what specific inputs get sent to a given tool call, a properly well-designed server needs to carefully validate those inputs itself, on its own end, rather than simply, blindly trusting that whatever the model happens to send along will always necessarily be safe, well-formed, and appropriate. This connects directly back to the malicious-prompt-injection risk we've touched on before in earlier discussions — if an AI model can be tricked or manipulated, whether through cleverly crafted user input or through poisoned, adversarial external content it happens to read, into requesting some genuinely harmful action through a tool call, a properly secure server needs real, robust safeguards in place that don't rely purely and solely on the AI model itself always behaving correctly and never making a mistake.

There's also a genuinely important, broader trust question specifically worth understanding: because MCP servers can, in principle, be built and published by literally anyone, connecting your own AI application to some random, completely unfamiliar, unvetted third-party MCP server inherently and necessarily carries some real, non-trivial risk — you're effectively granting that specific server's code some meaningful degree of access and trust. Just like installing an unfamiliar, unvetted app on your own personal phone carries some real, inherent risk, connecting to an unfamiliar, unvetted MCP server carries a genuinely similar and comparable kind of risk, which is exactly why using MCP servers specifically from trusted, well-established, reputable sources matters quite a lot in practice, rather than casually and indiscriminately connecting to just any random, unfamiliar server one happens to come across.

13 Enterprise MCP Design

Enterprise MCP design refers to the additional set of considerations that genuinely come into play specifically when a large organization wants to deploy and actually use MCP at real, meaningful scale, across many different internal teams, systems, and users, rather than just one individual developer casually experimenting and connecting to a small handful of tools purely for their own personal, individual use.

A few particular considerations become genuinely more important at this larger, more organizational scale. **Centralized governance** matters a great deal — a large organization generally wants some genuine, deliberate control and clear, meaningful visibility over exactly which specific MCP servers its employees are actually allowed to connect to in the first place, rather than each individual employee independently and informally connecting to whatever random, unvetted servers they personally happen to come across on their own. **Access control that's properly scoped to each specific individual user** matters too — a genuinely well-designed enterprise MCP setup ensures that a given specific employee, when connecting through the organization's systems, can only actually access the specific particular data and specific particular tools they're genuinely authorized to use for their own particular role, rather than every single connected user automatically getting identical, blanket access to absolutely everything a given server might happen to expose. **Auditability** matters quite a lot as well — being able to reliably track and clearly review exactly which tools were actually called, by which specific user, and when, becomes genuinely important for both meaningful security oversight and broader regulatory compliance purposes, especially in larger, more heavily regulated organizations and industries. And **reliability and genuine scalability** matter too — an MCP server that's meant to reliably serve an entire large organization, rather than just one individual person working alone, needs to be properly built to reliably handle a meaningfully higher, more sustained volume of simultaneous connections and simultaneous requests without breaking down or becoming unreliable under that increased real-world load.

Taken together, these enterprise-specific considerations reflect a now-familiar, recurring theme we've seen echoed throughout this whole series — the same fundamental core technology or protocol that works perfectly well for a small, simple, individual prototype or experiment often needs real, genuine additional engineering discipline and care layered carefully on top of it before it's actually genuinely ready and robust enough for reliable, safe, large-scale production use within a real, serious organization.